

LAPORAN PRAKTIKUM

INTERNET OF THINGS II

Praktikum 2: Antarmuka Sensor dan Sistem Kendali Dasar



Di susun oleh:

ANDREAN FIRNANDO (25782090)

TRI – 3D

Dosen Pengajar :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M. T. Agus

Ambarwari, S.Pd., M.kom

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027

Praktikum 1

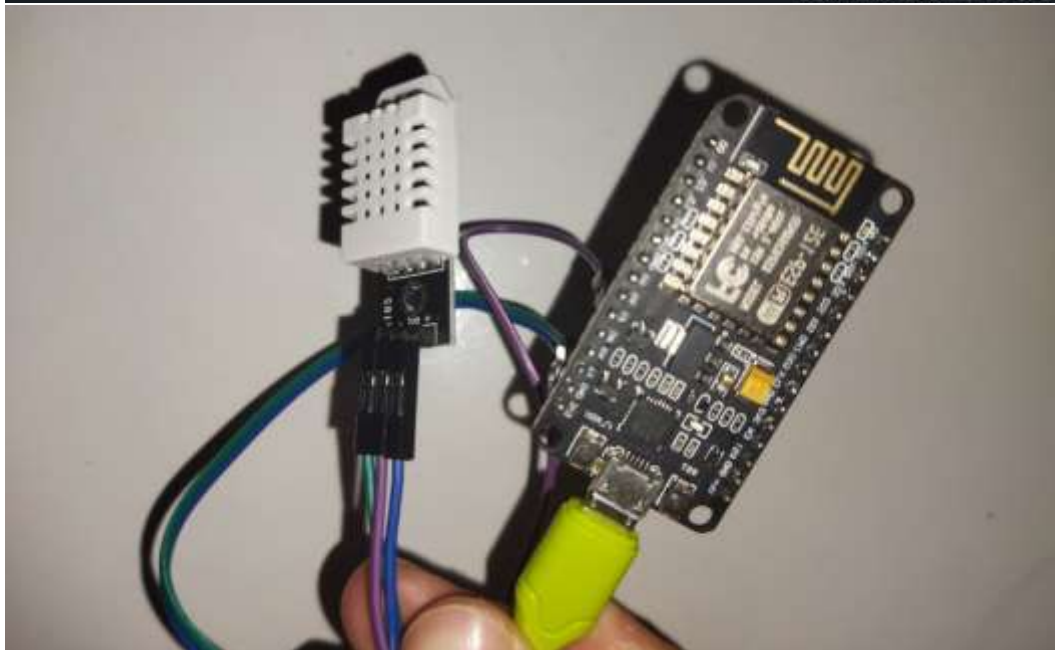
The screenshot shows the NodeMCU IDE with a C program for SPI communication. The program defines a pin (14), an array of data, and a function to write data to the SPI bus. The serial monitor shows the output of the program, displaying the data being written to the SPI bus.

```

1 // Pin 14 is the SPI MISO pin
2
3 #include <SPI.h>
4
5 // Data to be sent
6 const uint8_t data[] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10};
7
8 // SPI MISO pin
9 const int MISO_PIN = 14;
10
11 // SPI MISO pin
12 const int MISO_PIN = 14;
13
14 // SPI MISO pin
15 const int MISO_PIN = 14;
16
17 // SPI MISO pin
18 const int MISO_PIN = 14;
19
20 // SPI MISO pin
21 const int MISO_PIN = 14;
22
23 // SPI MISO pin
24 const int MISO_PIN = 14;
25
26 // SPI MISO pin
27 const int MISO_PIN = 14;
28
29 // SPI MISO pin
30 const int MISO_PIN = 14;
31
32 // SPI MISO pin
33 const int MISO_PIN = 14;
34
35 // SPI MISO pin
36 const int MISO_PIN = 14;
37
38 // SPI MISO pin
39 const int MISO_PIN = 14;
40
41 // SPI MISO pin
42 const int MISO_PIN = 14;
43
44 // SPI MISO pin
45 const int MISO_PIN = 14;
46
47 // SPI MISO pin
48 const int MISO_PIN = 14;
49
50 // SPI MISO pin
51 const int MISO_PIN = 14;
52
53 // SPI MISO pin
54 const int MISO_PIN = 14;
55
56 // SPI MISO pin
57 const int MISO_PIN = 14;
58
59 // SPI MISO pin
60 const int MISO_PIN = 14;
61
62 // SPI MISO pin
63 const int MISO_PIN = 14;
64
65 // SPI MISO pin
66 const int MISO_PIN = 14;
67
68 // SPI MISO pin
69 const int MISO_PIN = 14;
70
71 // SPI MISO pin
72 const int MISO_PIN = 14;
73
74 // SPI MISO pin
75 const int MISO_PIN = 14;
76
77 // SPI MISO pin
78 const int MISO_PIN = 14;
79
80 // SPI MISO pin
81 const int MISO_PIN = 14;
82
83 // SPI MISO pin
84 const int MISO_PIN = 14;
85
86 // SPI MISO pin
87 const int MISO_PIN = 14;
88
89 // SPI MISO pin
90 const int MISO_PIN = 14;
91
92 // SPI MISO pin
93 const int MISO_PIN = 14;
94
95 // SPI MISO pin
96 const int MISO_PIN = 14;
97
98 // SPI MISO pin
99 const int MISO_PIN = 14;
100
101 // SPI MISO pin
102 const int MISO_PIN = 14;
103
104 // SPI MISO pin
105 const int MISO_PIN = 14;
106
107 // SPI MISO pin
108 const int MISO_PIN = 14;
109
110 // SPI MISO pin
111 const int MISO_PIN = 14;
112
113 // SPI MISO pin
114 const int MISO_PIN = 14;
115
116 // SPI MISO pin
117 const int MISO_PIN = 14;
118
119 // SPI MISO pin
120 const int MISO_PIN = 14;
121
122 // SPI MISO pin
123 const int MISO_PIN = 14;
124
125 // SPI MISO pin
126 const int MISO_PIN = 14;
127
128 // SPI MISO pin
129 const int MISO_PIN = 14;
130
131 // SPI MISO pin
132 const int MISO_PIN = 14;
133
134 // SPI MISO pin
135 const int MISO_PIN = 14;
136
137 // SPI MISO pin
138 const int MISO_PIN = 14;
139
140 // SPI MISO pin
141 const int MISO_PIN = 14;
142
143 // SPI MISO pin
144 const int MISO_PIN = 14;
145
146 // SPI MISO pin
147 const int MISO_PIN = 14;
148
149 // SPI MISO pin
150 const int MISO_PIN = 14;
151
152 // SPI MISO pin
153 const int MISO_PIN = 14;
154
155 // SPI MISO pin
156 const int MISO_PIN = 14;
157
158 // SPI MISO pin
159 const int MISO_PIN = 14;
160
161 // SPI MISO pin
162 const int MISO_PIN = 14;
163
164 // SPI MISO pin
165 const int MISO_PIN = 14;
166
167 // SPI MISO pin
168 const int MISO_PIN = 14;
169
170 // SPI MISO pin
171 const int MISO_PIN = 14;
172
173 // SPI MISO pin
174 const int MISO_PIN = 14;
175
176 // SPI MISO pin
177 const int MISO_PIN = 14;
178
179 // SPI MISO pin
180 const int MISO_PIN = 14;
181
182 // SPI MISO pin
183 const int MISO_PIN = 14;
184
185 // SPI MISO pin
186 const int MISO_PIN = 14;
187
188 // SPI MISO pin
189 const int MISO_PIN = 14;
190
191 // SPI MISO pin
192 const int MISO_PIN = 14;
193
194 // SPI MISO pin
195 const int MISO_PIN = 14;
196
197 // SPI MISO pin
198 const int MISO_PIN = 14;
199
200 // SPI MISO pin
201 const int MISO_PIN = 14;
202
203 // SPI MISO pin
204 const int MISO_PIN = 14;
205
206 // SPI MISO pin
207 const int MISO_PIN = 14;
208
209 // SPI MISO pin
210 const int MISO_PIN = 14;
211
212 // SPI MISO pin
213 const int MISO_PIN = 14;
214
215 // SPI MISO pin
216 const int MISO_PIN = 14;
217
218 // SPI MISO pin
219 const int MISO_PIN = 14;
220
221 // SPI MISO pin
222 const int MISO_PIN = 14;
223
224 // SPI MISO pin
225 const int MISO_PIN = 14;
226
227 // SPI MISO pin
228 const int MISO_PIN = 14;
229
230 // SPI MISO pin
231 const int MISO_PIN = 14;
232
233 // SPI MISO pin
234 const int MISO_PIN = 14;
235
236 // SPI MISO pin
237 const int MISO_PIN = 14;
238
239 // SPI MISO pin
240 const int MISO_PIN = 14;
241
242 // SPI MISO pin
243 const int MISO_PIN = 14;
244
245 // SPI MISO pin
246 const int MISO_PIN = 14;
247
248 // SPI MISO pin
249 const int MISO_PIN = 14;
250
251 // SPI MISO pin
252 const int MISO_PIN = 14;
253
254 // SPI MISO pin
255 const int MISO_PIN = 14;
256
257 // SPI MISO pin
258 const int MISO_PIN = 14;
259
260 // SPI MISO pin
261 const int MISO_PIN = 14;
262
263 // SPI MISO pin
264 const int MISO_PIN = 14;
265
266 // SPI MISO pin
267 const int MISO_PIN = 14;
268
269 // SPI MISO pin
270 const int MISO_PIN = 14;
271
272 // SPI MISO pin
273 const int MISO_PIN = 14;
274
275 // SPI MISO pin
276 const int MISO_PIN = 14;
277
278 // SPI MISO pin
279 const int MISO_PIN = 14;
280
281 // SPI MISO pin
282 const int MISO_PIN = 14;
283
284 // SPI MISO pin
285 const int MISO_PIN = 14;
286
287 // SPI MISO pin
288 const int MISO_PIN = 14;
289
290 // SPI MISO pin
291 const int MISO_PIN = 14;
292
293 // SPI MISO pin
294 const int MISO_PIN = 14;
295
296 // SPI MISO pin
297 const int MISO_PIN = 14;
298
299 // SPI MISO pin
300 const int MISO_PIN = 14;
301
302 // SPI MISO pin
303 const int MISO_PIN = 14;
304
305 // SPI MISO pin
306 const int MISO_PIN = 14;
307
308 // SPI MISO pin
309 const int MISO_PIN = 14;
310
311 // SPI MISO pin
312 const int MISO_PIN = 14;
313
314 // SPI MISO pin
315 const int MISO_PIN = 14;
316
317 // SPI MISO pin
318 const int MISO_PIN = 14;
319
320 // SPI MISO pin
321 const int MISO_PIN = 14;
322
323 // SPI MISO pin
324 const int MISO_PIN = 14;
325
326 // SPI MISO pin
327 const int MISO_PIN = 14;
328
329 // SPI MISO pin
330 const int MISO_PIN = 14;
331
332 // SPI MISO pin
333 const int MISO_PIN = 14;
334
335 // SPI MISO pin
336 const int MISO_PIN = 14;
337
338 // SPI MISO pin
339 const int MISO_PIN = 14;
340
341 // SPI MISO pin
342 const int MISO_PIN = 14;
343
344 // SPI MISO pin
345 const int MISO_PIN = 14;
346
347 // SPI MISO pin
348 const int MISO_PIN = 14;
349
350 // SPI MISO pin
351 const int MISO_PIN = 14;
352
353 // SPI MISO pin
354 const int MISO_PIN = 14;
355
356 // SPI MISO pin
357 const int MISO_PIN = 14;
358
359 // SPI MISO pin
360 const int MISO_PIN = 14;
361
362 // SPI MISO pin
363 const int MISO_PIN = 14;
364
365 // SPI MISO pin
366 const int MISO_PIN = 14;
367
368 // SPI MISO pin
369 const int MISO_PIN = 14;
370
371 // SPI MISO pin
372 const int MISO_PIN = 14;
373
374 // SPI MISO pin
375 const int MISO_PIN = 14;
376
377 // SPI MISO pin
378 const int MISO_PIN = 14;
379
380 // SPI MISO pin
381 const int MISO_PIN = 14;
382
383 // SPI MISO pin
384 const int MISO_PIN = 14;
385
386 // SPI MISO pin
387 const int MISO_PIN = 14;
388
389 // SPI MISO pin
390 const int MISO_PIN = 14;
391
392 // SPI MISO pin
393 const int MISO_PIN = 14;
394
395 // SPI MISO pin
396 const int MISO_PIN = 14;
397
398 // SPI MISO pin
399 const int MISO_PIN = 14;
400
401 // SPI MISO pin
402 const int MISO_PIN = 14;
403
404 // SPI MISO pin
405 const int MISO_PIN = 14;
406
407 // SPI MISO pin
408 const int MISO_PIN = 14;
409
410 // SPI MISO pin
411 const int MISO_PIN = 14;
412
413 // SPI MISO pin
414 const int MISO_PIN = 14;
415
416 // SPI MISO pin
417 const int MISO_PIN = 14;
418
419 // SPI MISO pin
420 const int MISO_PIN = 14;
421
422 // SPI MISO pin
423 const int MISO_PIN = 14;
424
425 // SPI MISO pin
426 const int MISO_PIN = 14;
427
428 // SPI MISO pin
429 const int MISO_PIN = 14;
430
431 // SPI MISO pin
432 const int MISO_PIN = 14;
433
434 // SPI MISO pin
435 const int MISO_PIN = 14;
436
437 // SPI MISO pin
438 const int MISO_PIN = 14;
439
440 // SPI MISO pin
441 const int MISO_PIN = 14;
442
443 // SPI MISO pin
444 const int MISO_PIN = 14;
445
446 // SPI MISO pin
447 const
```



Praktikum 2

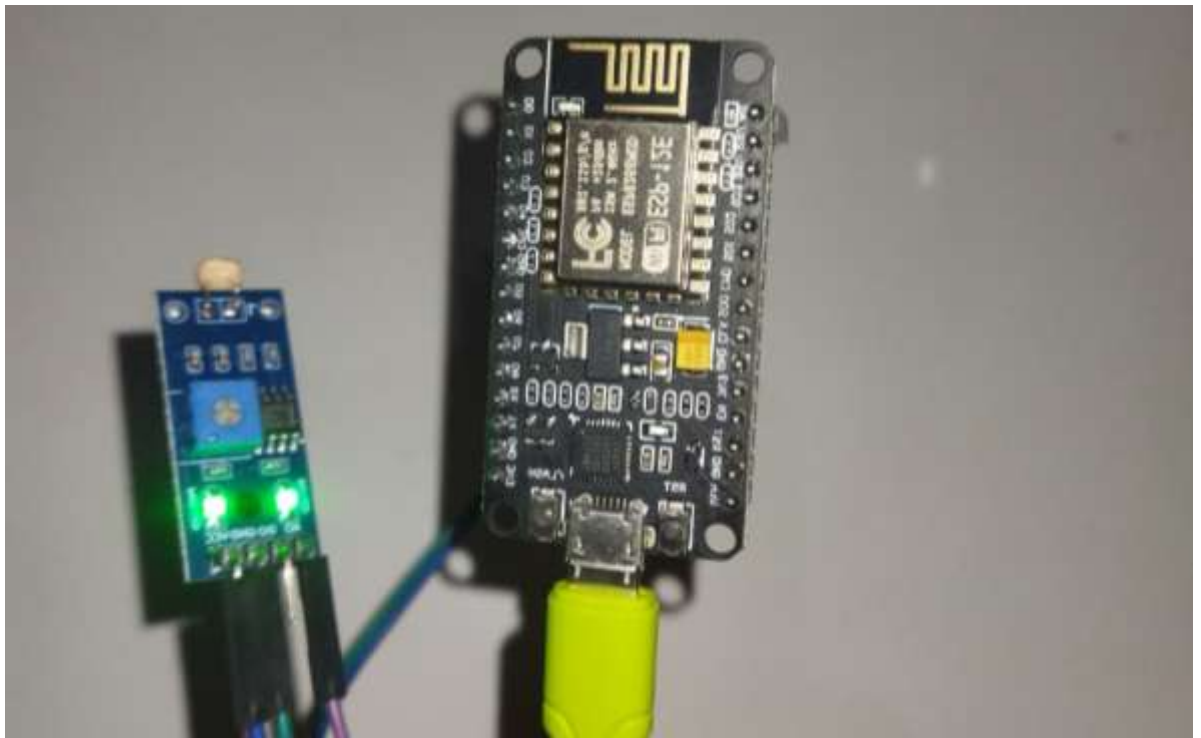
[illegible]

Latihan 1 (Pemahaman Sistem ADC): Mengapa batas maksimal nilai intensitas cahaya yang terbaca pada Serial Monitor di Praktikum 1 adalah sekitar angka 1023 (bukan 100 atau 1000)? Kaitkan jawaban Anda dengan penjelasan "Resolusi ADC 10-bit".

Karena spesifikasi mikrokontroler, ADC-nya memiliki resolusi 10-bit. Karena perhitungan angka dalam sistem komputasi dimulai dari angka 0, maka rentang nilai digital yang dihasilkan berkisar dari 0 hingga 1023 (total 1024 tingkat nilai). Angka 0 mewakili tegangan terendah (0V), sedangkan 1023 mewakili tegangan maksimum input analog (3.3V).

Latihan 2 (Implementasi Algoritma): Nilai ADC 0-1023 tidak lazim dibaca oleh pengguna akhir (manusia). Modifikasi kode pada Praktikum 1 agar program menghitung dan mencetak persentase intensitas cahaya (rentang 0% hingga 100%). Tuliskan rumus matematika / perintah pembagian yang Anda sematkan di kode tersebut!

[illegible]



$\text{Kode} = \text{float persentaseCahaya} = (\text{ldrValue} / 1023.0) * 100.0;$ Untuk mengubah skala linear 0-1023 menjadi skala persentase 0-100, kita menggunakan perbandingan senilai, Nilai bacaan aktual dibagi dengan batas maksimum skala ADC, lalu dikalikan dengan 100.

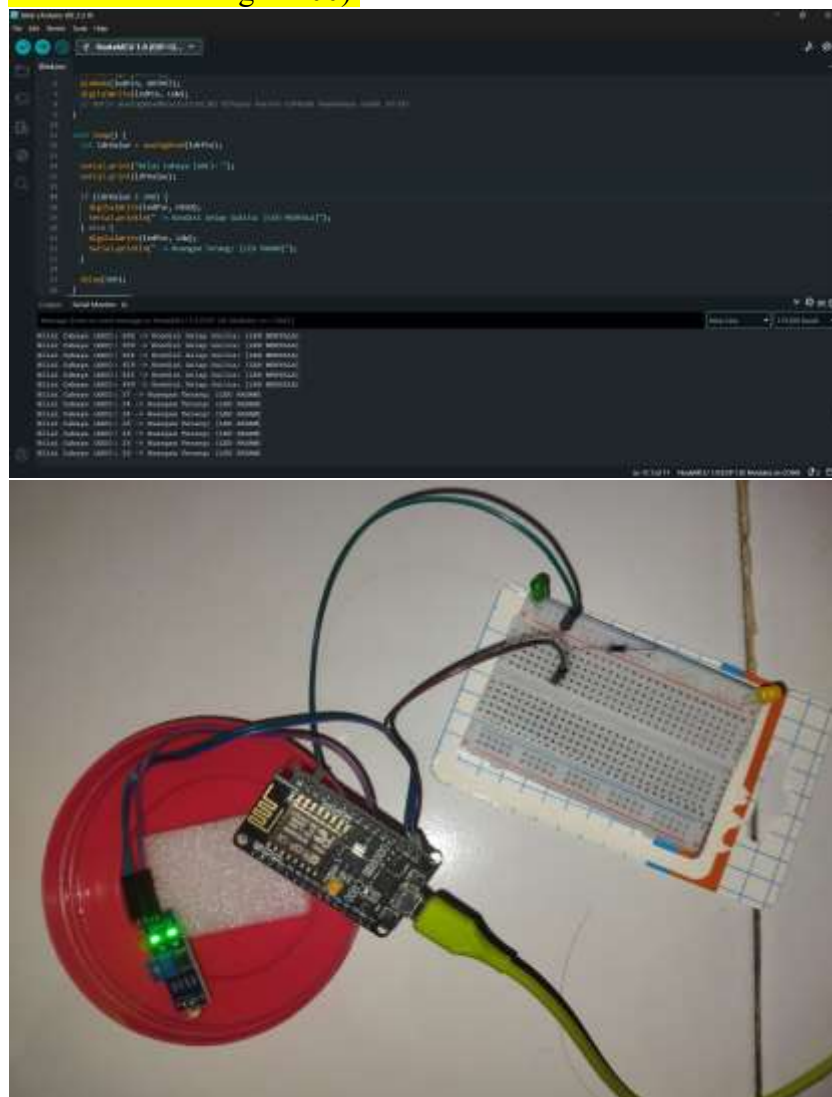
Latihan 3 (Analisis Rangkaian): Di subbab 2.3.4 disebutkan sifat modul *Relay* yang terisolasi *Optocoupler*. Banyak *Relay* di pasaran yang bekerja secara **Active-Low**. Jika *Relay* Anda bertipe *Active-Low*, apa yang terjadi pada aktuatur (misal: lampu ruangan) yang terhubung ke *Relay* jika program Anda mengeksekusi `digitalWrite(relayPin, HIGH)`?

Peran Optocoupler dan Isolasi Elektrik: Modul relay dilengkapi dengan optocoupler untuk memisahkan secara total sirkuit tegangan rendah mikrokontroler (3.3V) dengan sirkuit beban luar bertegangan tinggi seperti AC 220V PLN, ini melindungi mikrokontroler dari lonjakan arus. Jika program mengeksekusi perintah `digitalWrite(relayPin, HIGH)`, maka pin mikrokontroler akan mengeluarkan tegangan penuh (mendekati 3.3V atau 5V). Akibatnya, tuas

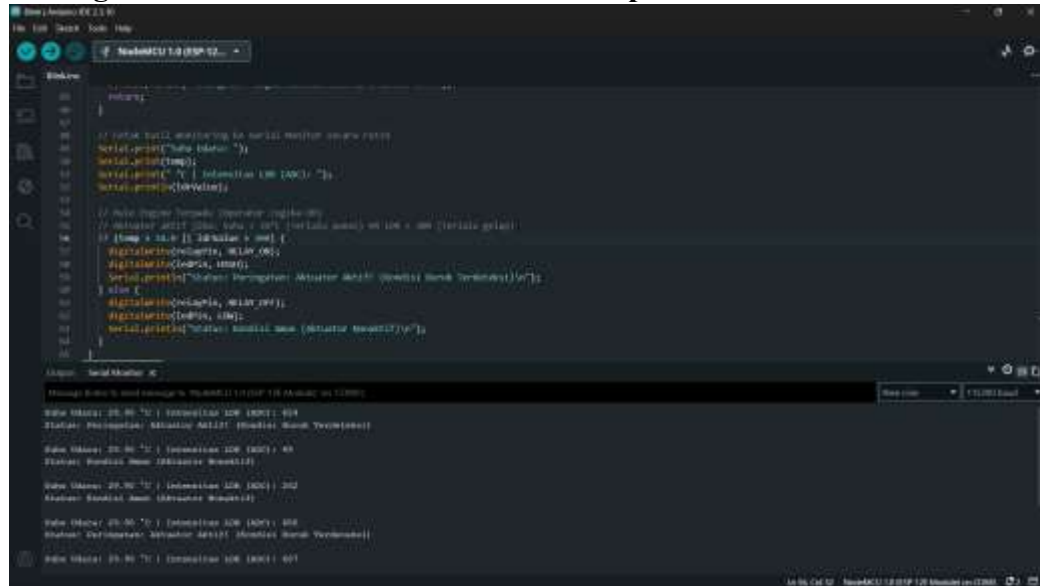
sakelar kembali ke posisi normal/terputus, dan aktuator/lampu ruangan yang terhubung dipastikan berada dalam kondisi mati (off).

Latihan 4 (Pengembangan Logika): Gabungkan wawasan dari Praktikum 1 Modul 2 ini dengan logika kendali LED dari Praktikum Modul 1. Gunakan blok kondisional if yang berfungsi menyalakan sebuah LED (D1) **HANYA JIKA** sensor LDR mendeteksi lingkungan yang gelap gulita (misalnya: ambang batas ADC berada di bawah angka 200).

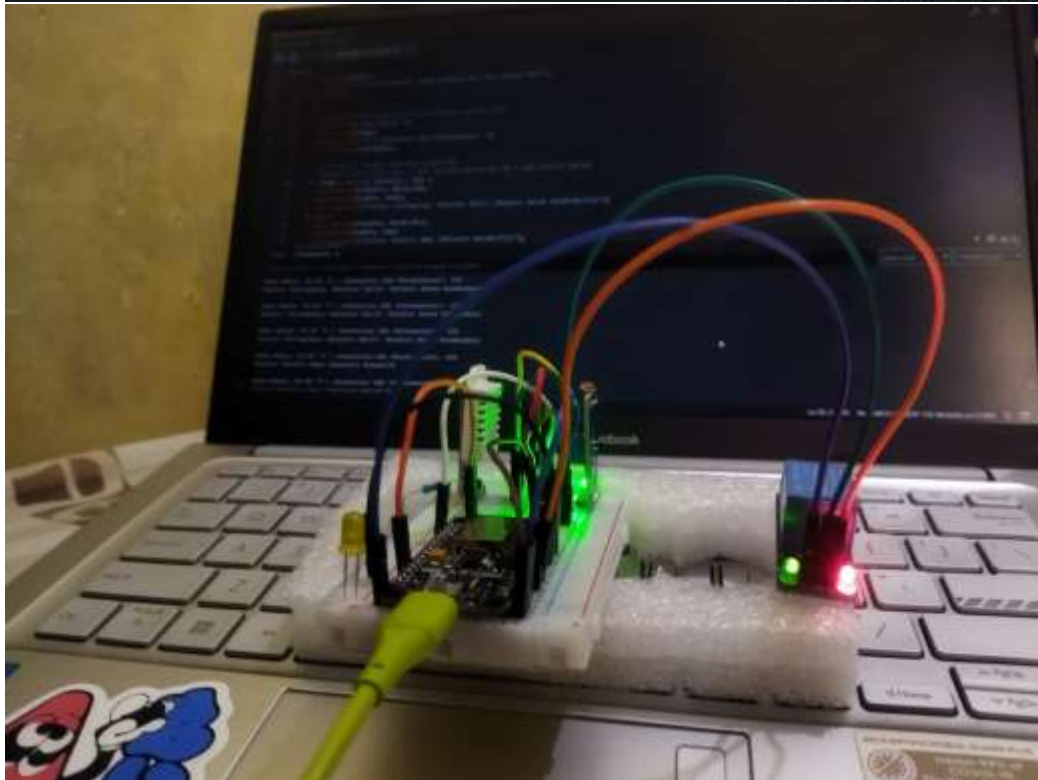
Lampu LED hanya akan dihidupkan apabila kondisi lingkungan memenuhi syarat mutlak, yaitu tingkat kegelapan yang pekat (nilai pembacaan ADC di bawah angka 200).



2.7 Tugas: Sistem Otomasi Kontrol Lokal Terpadu



```
1 // Sistem Otomasi Kontrol Lokal Terpadu
2 //
3 // Sistem ini berfungsi untuk memantau suhu dan kelembaban gudang menggunakan sensor DHT22, serta mengontrol lampu LED dan relay berdasarkan kondisi yang terdeteksi.
4 //
5 // Komponen yang digunakan:
6 // - NodeMCU ESP8266
7 // - Sensor DHT22 (Suhu & Kelembaban)
8 // - Modul LDR (Intensitas Cahaya)
9 // - Modul Relay
10 // - LED
11 //
12 // Logika Kontrol:
13 // - Jika suhu > 30°C atau kelembaban > 80%, maka LED akan menyala dan relay akan aktif.
14 // - Jika suhu < 30°C dan kelembaban < 80%, maka LED akan mati dan relay akan nonaktif.
15 //
16 //
17 //
18 //
19 //
20 //
21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96 //
97 //
98 //
99 //
100 //
```



Pada Praktikum Tugas Modul 2 berhasil merealisasikan prototipe IoT berbasis ESP8266 dengan mengintegrasikan sensor DHT22, Modul LDR, LED, dan Relay. Logika kontrol diprogram secara mandiri di tingkat *edge*: ketika kondisi bahaya terdeteksi (suhu tinggi atau intensitas cahaya rendah), NodeMCU akan memberikan sinyal kontrol untuk mengaktifkan beban Relay dan LED indikator visual secara bersamaan, serta menonaktifkannya kembali saat lingkungan gudang berada pada parameter yang aman.